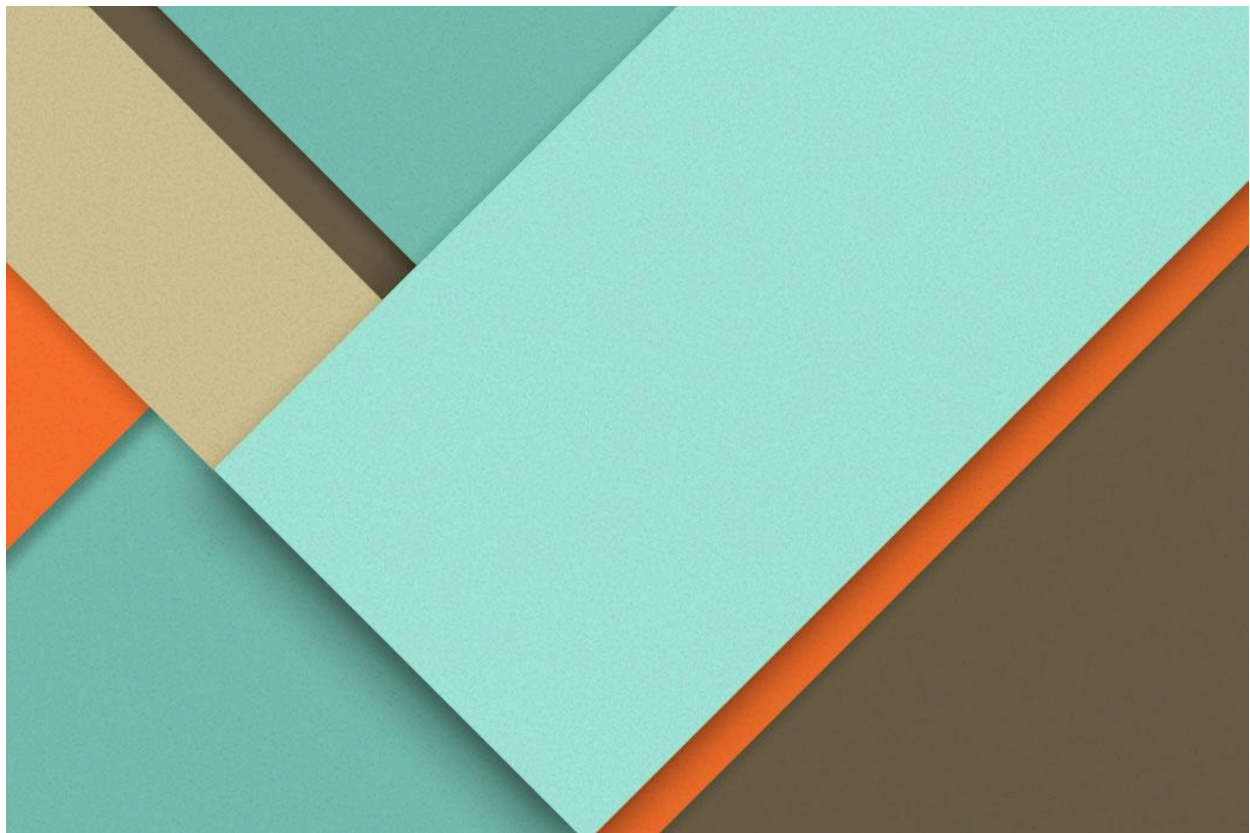




# Prueba técnica:

Pipeline de transformación con dbt por capas con vinculación a BigQuery

14/05/2026

Monterrey N.L.

—

Por Brandon Marquez Figueroa

Challenge/Prueba técnica Manuable

## Contexto del Negocio

Eres parte del equipo de BI de una empresa de e-commerce. Se te proporciona acceso a un proyecto en BigQuery con los siguientes datos crudos en el esquema **raw**:

- raw.orders — pedidos realizados
- raw.order\_items — ítems por pedido
- raw.customers — clientes
- raw.products — catálogo de productos

Los datos tienen problemas de calidad: campos nulos, fechas mal formateadas, IDs duplicados y registros inconsistentes. Tu misión es construir un pipeline de transformación con dbt que entregue datos confiables y listos para análisis.

## Entregables

Se evaluarán **4 secciones** con los siguientes pesos:

| Sección | Tema                     | Puntos |
|---------|--------------------------|--------|
| 1       | Modelado con dbt         | 40     |
| 2       | Calidad de datos & tests | 25     |
| 3       | Orquestación con Airflow | 20     |
| 4       | Análisis & documentación | 15     |
| Total   |                          | 100    |

## Pasos preliminares

- Creación y configuración del proyecto en BigQuery.
- Creación de cuenta de servicio con los permisos correspondientes
- Generación de archivo con credenciales de acceso tipo json.

Google Cloud **dbt-challenge-manuable** Search (/) for resources, docs, products, and more Search

IAM & Admin / IAM

**IAM**

Allow Deny Recommendations history

View by principals View by roles

Grant access Remove access

Filter Enter property name or value

| Type                     | Principal                                                          | Name                     | Role                                       | Security insights         |
|--------------------------|--------------------------------------------------------------------|--------------------------|--------------------------------------------|---------------------------|
| <input type="checkbox"/> | dbt-service-account@dbt-challenge-manuable.iam.gserviceaccount.com | dbt-service-account      | Access Transparency Admin                  | Advanced security insight |
|                          |                                                                    |                          | BigQuery Data Editor                       | Advanced security insight |
|                          |                                                                    |                          | BigQuery Job User                          | Advanced security insight |
|                          |                                                                    |                          | BigQuery User                              | Advanced security insight |
|                          |                                                                    |                          | Compute Organization Security Policy Admin | Advanced security insight |
|                          |                                                                    |                          | Organization Administrator                 | Advanced security insight |
|                          |                                                                    |                          | Organization Policy Viewer                 | Advanced security insight |
|                          |                                                                    |                          | Owner                                      |                           |
| <input type="checkbox"/> | mar.zignas@gmail.com                                               | Brandon Marquez Figueroa | Access Transparency Admin                  | Advanced security insight |
|                          |                                                                    |                          | BigQuery Data Editor                       | Advanced security insight |

```
Credentials > {} dbt-challenge-manuable-f640869813f5.json > ...
1  {
2    "type": "service_account",
3    "project_id": "dbt-challenge-manuable",
4    "private_key_id": "f640869813f54dff8be28e1d148475373058ff93",
5    "private_key": "-----BEGIN PRIVATE KEY-----\nMIIEvQIBADANBgkqhkiG9w0BAQEFAASCBCwggSjAgEAAoIBAQC
6    "client_email": "dbt-service-account@dbt-challenge-manuable.iam.gserviceaccount.com",
7    "client_id": "114877742749696545411",
8    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
9    "token_uri": "https://oauth2.googleapis.com/token",
10   "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
11   "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/dbt-service-account%4
12   "universe_domain": "googleapis.com"
13 }
14
```

- Instalación de dbt y configuración para BigQuery

**Nota:** Requiere ajustes de compatibilidad, para instalación por pip de python se requiere una versión no más reciente que Python 3.11.x

```

TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  PORTS
C:\Users\mar_z\Desktop\dbt-challenge-manuable>dbt --version
Core:
- installed: 1.11.9
- latest:    1.11.10 - Update available!

Your version of dbt-core is out of date!
You can find instructions for upgrading here:
https://docs.getdbt.com/docs/installation

Plugins:
- bigquery: 1.11.1 - Up to date!

```

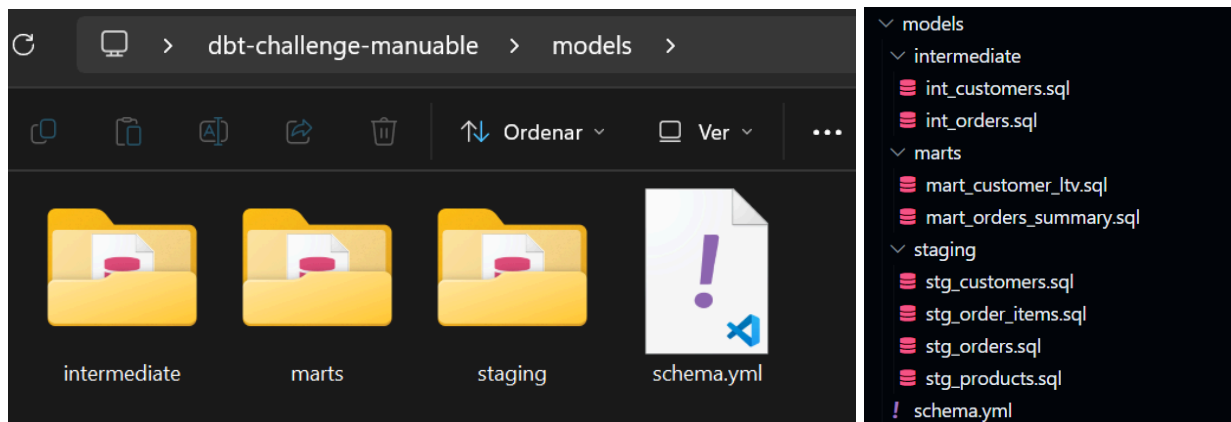
## Sección 1 – Modelado con dbt (40 pts)

### 1.1 Estructura de capas (10 pts)

Crea un proyecto dbt con la siguiente estructura de capas:

models/

- staging/ ← limpieza 1:1 de tablas raw
- intermediate/ ← lógica de negocio
- marts/ ← tablas finales para consumo



- 4 modelos staging (uno por tabla raw)
- 2 modelos intermediate
- 2 modelos en marts

## 1.2 Modelos requeridos (20 pts)

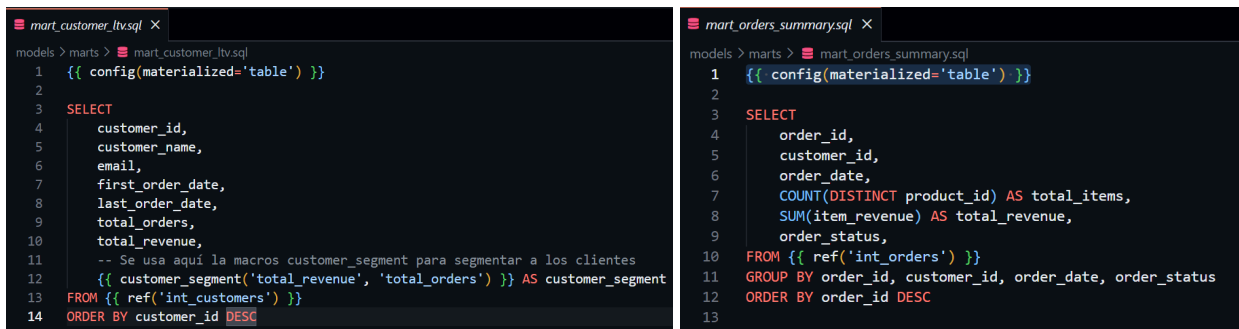
Construye los siguientes modelos en la capa marts:

**mart\_orders\_summary** — Una fila por orden, con:

- order\_id, customer\_id, order\_date
- total\_items (cantidad de items)
- total\_revenue (suma del precio × cantidad)
- order\_status normalizado (en minúsculas, sin espacios)

**mart\_customer\_ltv** — Una fila por cliente, con:

- customer\_id, customer\_name, email
- first\_order\_date, last\_order\_date
- total\_orders, total\_revenue
- customer\_segment: 'VIP' si revenue > \$5,000, 'Regular' si revenue > \$500, 'Nuevo' si tiene menos de 2 órdenes



```

mart_customer_ltv.sql
models > marts > mart_customer_ltv.sql
1  {{ config(materialized='table') }}
2
3  SELECT
4      customer_id,
5      customer_name,
6      email,
7      first_order_date,
8      last_order_date,
9      total_orders,
10     total_revenue,
11     -- Se usa aquí la macros customer_segment para segmentar a los clientes
12     {{ customer_segment('total_revenue', 'total_orders') }} AS customer_segment
13 FROM {{ ref('int_customers') }}
14 ORDER BY customer_id DESC

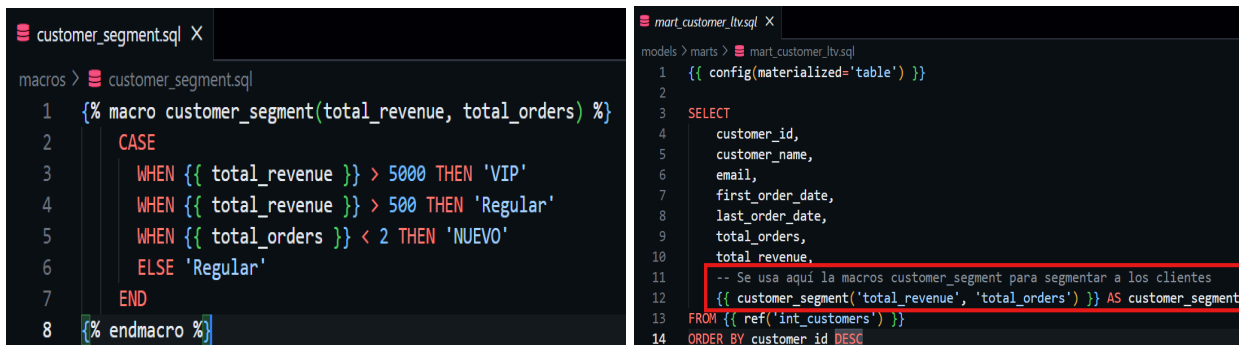
mart_orders_summary.sql
models > marts > mart_orders_summary.sql
1  {{ config(materialized='table') }}
2
3  SELECT
4      order_id,
5      customer_id,
6      order_date,
7      COUNT(DISTINCT product_id) AS total_items,
8      SUM(item_revenue) AS total_revenue,
9      order_status,
10 FROM {{ ref('int_orders') }}
11 GROUP BY order_id, customer_id, order_date, order_status
12 ORDER BY order_id DESC
13
  
```

## 1.3 Macros y reutilización (10 pts)

Crea al menos una macro dbt reutilizable. Sugerencias:

- Macro para normalizar texto (trim, lower)
- Macro para calcular segmento de cliente
- Macro para aplicar un filtro de fechas estándar

Úsala en al menos dos modelos



```

customer_segment.sql
macros > customer_segment.sql
1  {% macro customer_segment(total_revenue, total_orders) %}
2      CASE
3          WHEN {{ total_revenue }} > 5000 THEN 'VIP'
4          WHEN {{ total_revenue }} > 500 THEN 'Regular'
5          WHEN {{ total_orders }} < 2 THEN 'NUEVO'
6          ELSE 'Regular'
7      END
8  {% endmacro %}

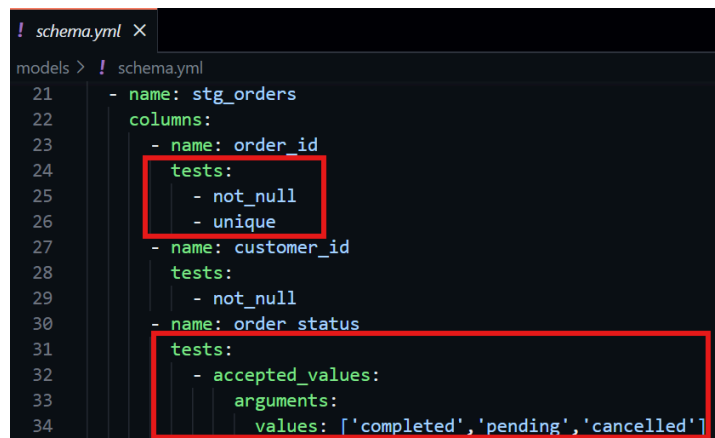
mart_customer_ltv.sql
models > marts > mart_customer_ltv.sql
1  {{ config(materialized='table') }}
2
3  SELECT
4      customer_id,
5      customer_name,
6      email,
7      first_order_date,
8      last_order_date,
9      total_orders,
10     total_revenue,
11     -- Se usa aquí la macros customer_segment para segmentar a los clientes
12     {{ customer_segment('total_revenue', 'total_orders') }} AS customer_segment
13 FROM {{ ref('int_customers') }}
14 ORDER BY customer_id DESC
  
```

## Sección 2 – Calidad de datos & Tests (25 pts)

### 2.1 Tests genéricos (10 pts)

En el archivo schema.yml, aplica tests dbt a los modelos staging y marts:

- not\_null en columnas críticas
- unique en PKs
- accepted\_values donde corresponda (ej. order\_status)
- relationships entre tablas (foreign keys)



```

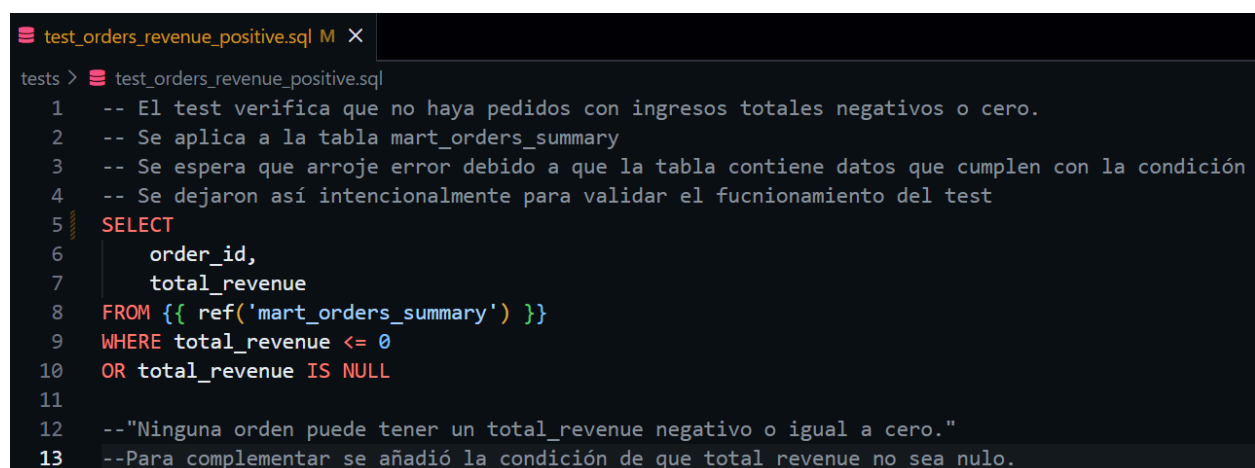
! schema.yml
models:
  - name: stg_orders
    columns:
      - name: order_id
        tests:
          - not_null
          - unique
      - name: customer_id
        tests:
          - not_null
      - name: order_status
        tests:
          - accepted_values:
              arguments:
                values: ['completed', 'pending', 'cancelled']
  
```

### 2.2 Test singular personalizado (10 pts)

Crea un test singular en tests/ que valide la siguiente regla de negocio:

"Ninguna orden puede tener un total\_revenue negativo o igual a cero."

El test debe fallar si encuentra registros que violen esta regla.

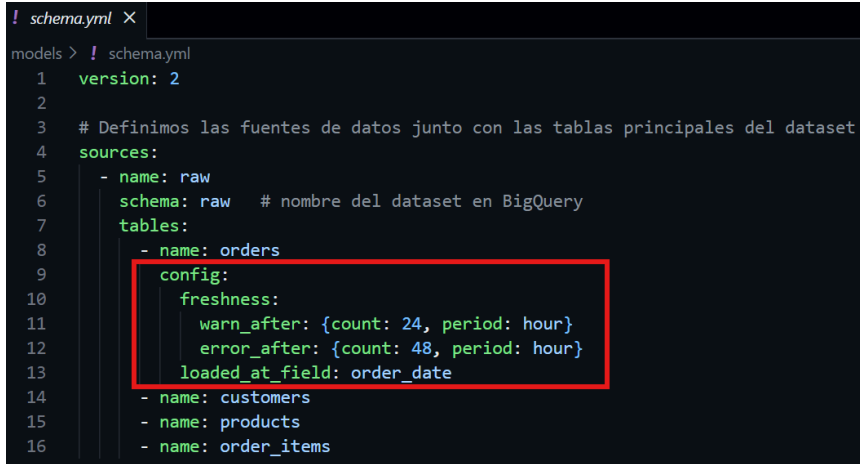


```

test_orders_revenue_positive.sql
-- El test verifica que no haya pedidos con ingresos totales negativos o cero.
-- Se aplica a la tabla mart_orders_summary
-- Se espera que arroje error debido a que la tabla contiene datos que cumplen con la condición
-- Se dejaron así intencionalmente para validar el funcionamiento del test
SELECT
  order_id,
  total_revenue
FROM {{ ref('mart_orders_summary') }}
WHERE total_revenue <= 0
OR total_revenue IS NULL
--"Ninguna orden puede tener un total_revenue negativo o igual a cero."
--Para complementar se añadió la condición de que total_revenue no sea nulo.
  
```

## 2.3 Source freshness (5 pts)

Configura sources.yml con las tablas raw y agrega configuración de freshness para alertar si los datos de raw.orders tienen más de 24 horas sin actualizarse.



```
! schema.yml X
models > ! schema.yml
1  version: 2
2
3  # Definimos las fuentes de datos junto con las tablas principales del dataset
4  sources:
5    - name: raw
6      schema: raw # nombre del dataset en BigQuery
7      tables:
8        - name: orders
9          config:
10             freshness:
11               warn_after: {count: 24, period: hour}
12               error_after: {count: 48, period: hour}
13               loaded_at_field: order_date
14        - name: customers
15        - name: products
16        - name: order_items
```

## Sección 3 – Orquestación con Airflow (20 pts)

### 3.1 DAG básico (15 pts)

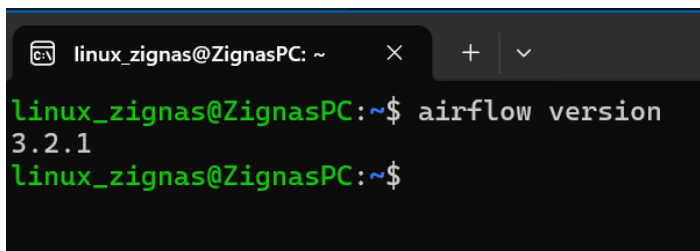
Crea un DAG en Airflow que:

1. Corra a las 2:00 AM todos los días
2. Ejecute dbt run --select staging primero
3. Luego ejecute dbt test --select staging
4. Si los tests pasan, ejecute dbt run --select marts
5. Finalmente ejecute dbt test --select marts

Puedes usar el operador BashOperator o DbtRunOperator del paquete astronomer-cosmos.

### Pasos previos

- Instalación WSL, Airflow requiere el entorno de linux, para este caso se instaló la versión 3.2.1 por temas de compatibilidad.

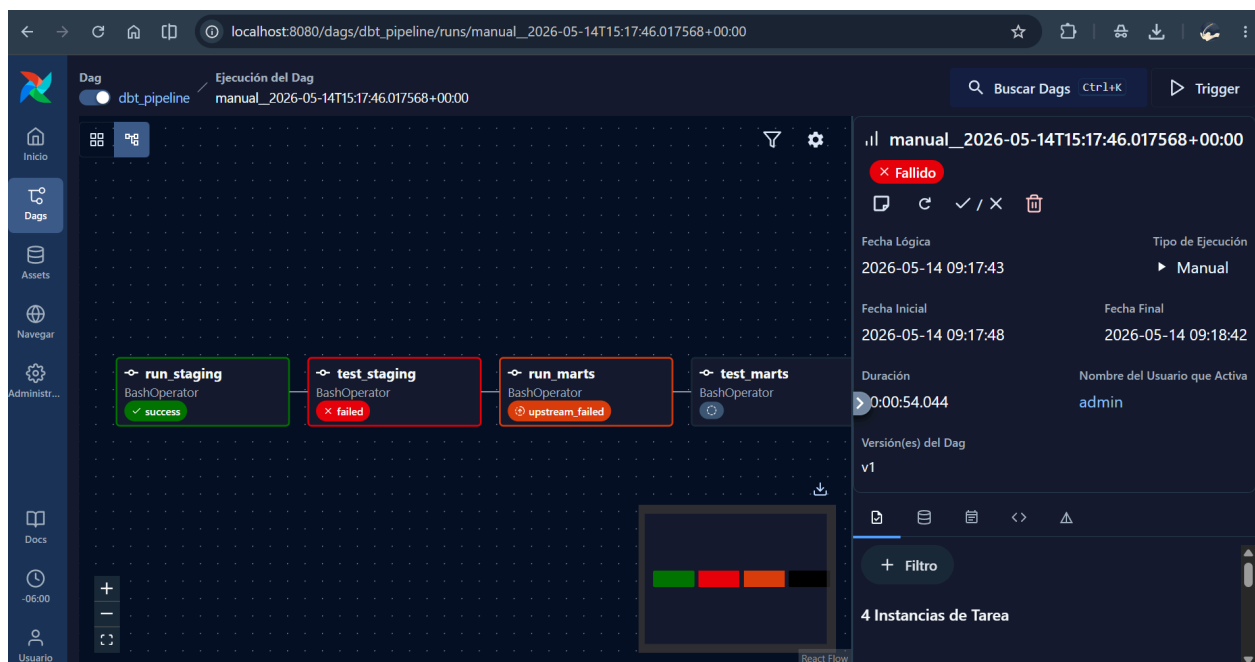


```
linux_zignas@ZignasPC: ~
linux_zignas@ZignasPC:~$ airflow version
3.2.1
linux_zignas@ZignasPC:~$
```

## Creación del archivo dbt\_pipeline.py

```
dbt_pipeline.py X
Ubuntu > home > linux_zignas > airflow > dags > dbt_pipeline.py > ...
1 from airflow import DAG
2 from airflow.providers.standard.operators.bash import BashOperator
3 from datetime import datetime
4
5 with DAG(
6     dag_id="dbt_pipeline",
7     start_date=datetime(2024, 1, 1),
8     schedule="0 2 * * *", # todos los días a las 2:00 AM
9     catchup=False,
10 ) as dag:
11
12     run_staging = BashOperator(
13         task_id="run_staging",
14         bash_command="cd /mnt/c/Users/mar_z/Desktop/dbt-challenge-manuable && dbt run --select sta
15     )
16
17     test_staging = BashOperator(
18         task_id="test_staging",
19         bash_command="cd /mnt/c/Users/mar_z/Desktop/dbt-challenge-manuable && dbt test --select st
```

Flujo en la interfaz de Airflow, El DAG debe tener manejo de errores: si dbt test falla, el pipeline debe detenerse y no continuar al siguiente paso.



### 3.2 Pregunta teórica (5 pts)

Responde en máximo 150 palabras:

¿Qué estrategia usarías para manejar tablas grandes en BigQuery (más de 500 millones de filas) dentro de tus modelos dbt? Menciona al menos dos conceptos específicos de dbt o BigQuery que aplicarías.

#### *Respuesta*

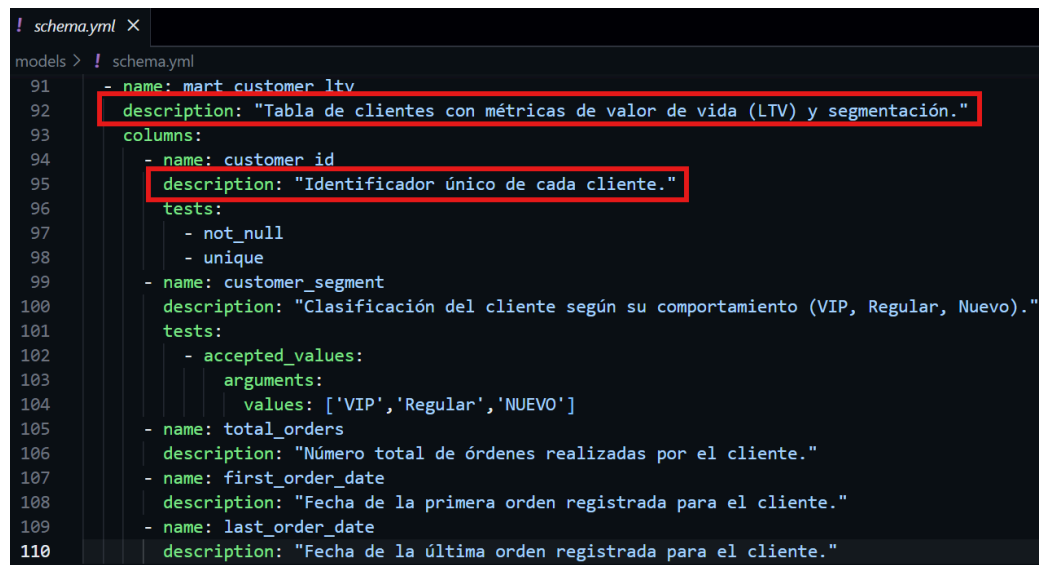
Al trabajar con tablas muy grandes en BigQuery (más de 500 millones de filas), la prioridad es reducir el volumen procesado en cada ejecución. Una primera estrategia es usar **modelos incrementales en dbt**, que actualizan solo los datos nuevos o modificados en lugar de recalculer toda la tabla completa. Esto evita escaneos innecesarios y hace que las cargas sean más rápidas y baratas. La segunda estrategia es **aprovechar las tablas particionadas y agrupadas en BigQuery**, que dividen la información por fechas o categorías y permiten consultar únicamente los segmentos relevantes. Además, conviene **crear etapas intermedias** con datos filtrados o resumidos, para que los modelos finales trabajen sobre conjuntos más pequeños y precisos. En conjunto, estas prácticas garantizan eficiencia, menor costo y escalabilidad en proyectos con datos masivos.

## Sección 4 – Análisis y Documentación (15 pts)

### 4.1 Documentación dbt (5 pts)

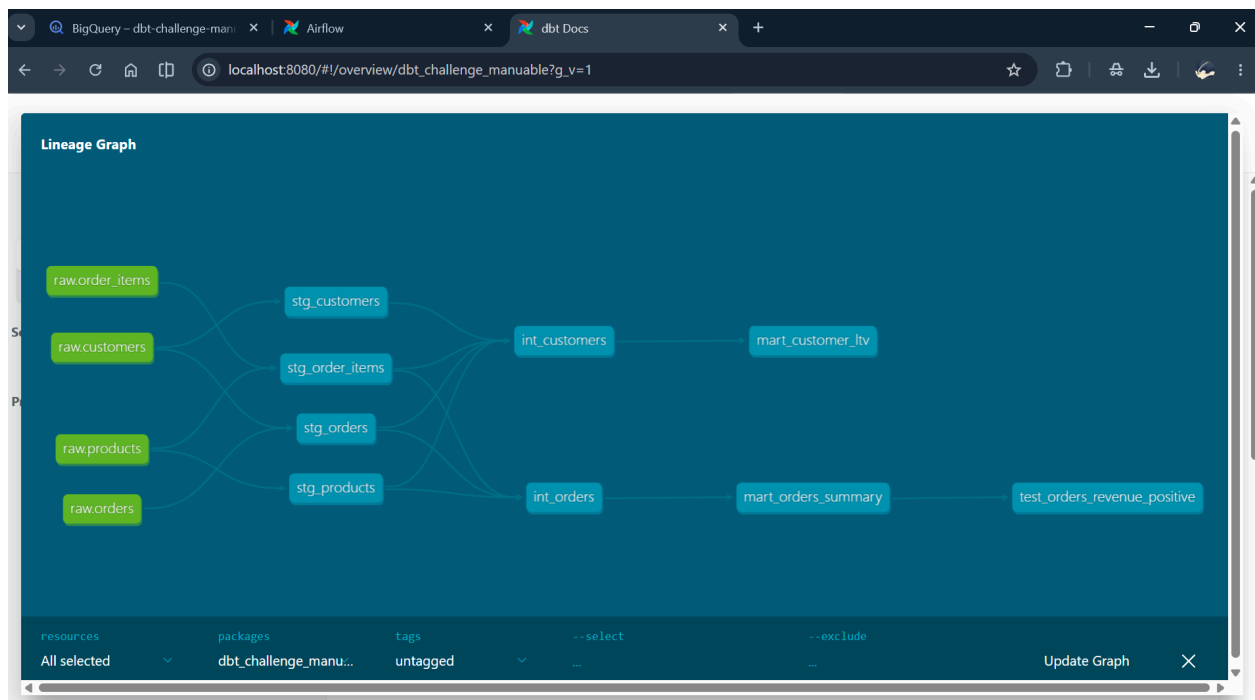
En el schema.yml, agrega:

- description en todos los modelos de marts
- description en al menos 5 columnas de mart\_customer\_ltv



```
! schema.yml X
models > ! schema.yml
91 - name: mart_customer_ltv
92   description: "Tabla de clientes con métricas de valor de vida (LTV) y segmentación."
93   columns:
94     - name: customer_id
95       description: "Identificador único de cada cliente."
96       tests:
97         - not_null
98         - unique
99     - name: customer_segment
100       description: "Clasificación del cliente según su comportamiento (VIP, Regular, Nuevo)."
101       tests:
102         - accepted_values:
103             arguments:
104               values: ['VIP', 'Regular', 'NUEVO']
105     - name: total_orders
106       description: "Número total de órdenes realizadas por el cliente."
107     - name: first_order_date
108       description: "Fecha de la primera orden registrada para el cliente."
109     - name: last_order_date
110       description: "Fecha de la última orden registrada para el cliente."
```

Ejecuta dbt docs generate y adjunta captura del lineage graph.



## 4.2 Consulta analítica en BigQuery (10 pts)

Escribe una query SQL directamente en BigQuery (sin dbt) que responda:  
 "¿Cuáles son los 5 productos más vendidos por mes durante el último trimestre, y cuál fue su variación porcentual en revenue vs el mes anterior?"  
 La query debe usar window functions (LAG, RANK o similar).

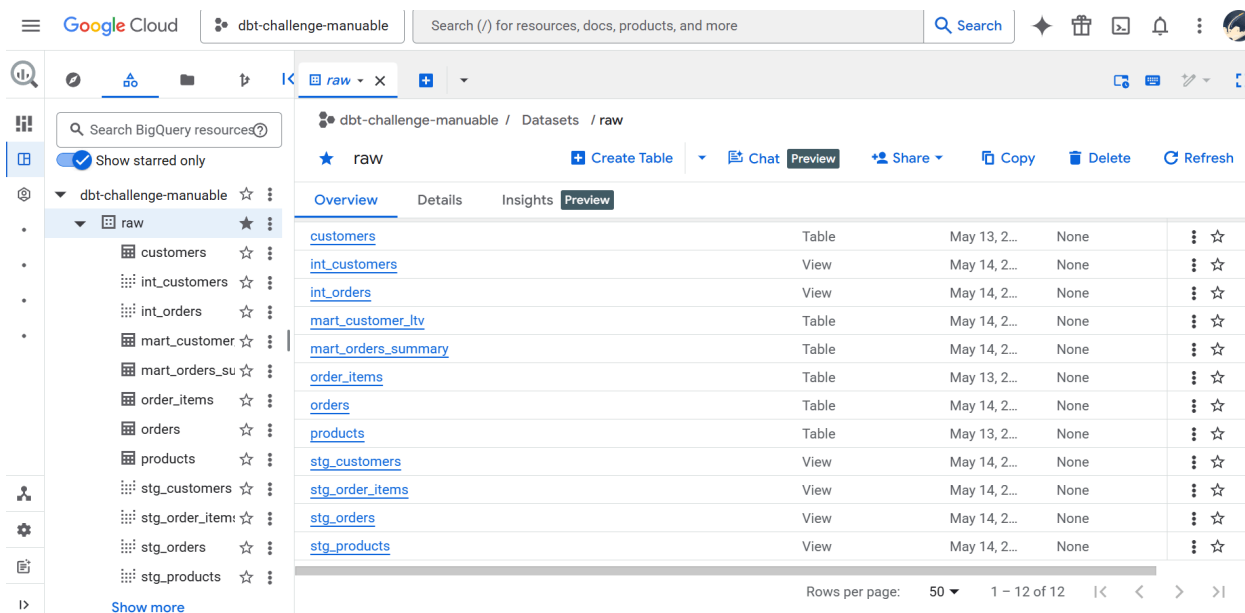
```

1 /*¿Cuáles son los 5 productos más vendidos por mes durante el último trimestre,
2 y cuál fue su variación porcentual en revenue vs el mes anterior?*/
3
4
5 -- Tabla temporal para extraer los registros de los últimos tres meses
6 WITH monthly_sales AS (
7   SELECT
8     FORMAT_DATE('%Y-%m', DATE(order_date)) AS month, --Ajuste de año con mes
9     product_id,
10    product_name,
11    SUM(quantity) AS total_quantity, -- Cantidad total de ventas por mes de cada producto
12    SUM(item_revenue) AS total_revenue -- Ingreso total de ventas por mes de cada producto
13  FROM raw.int_orders
14  WHERE order_date >= DATE_SUB(CURRENT_DATE(), INTERVAL 3 MONTH) -- Filtro del último trimestre
15        AND order_status = 'completed' -- Solo órdenes completadas
16        AND quantity IS NOT NULL -- Cantidades no nulas
17        AND quantity > 0 -- Cantidades no negativas
18  GROUP BY month, product_id, product_name
19 ),
20
21 -- Tabla temporal para obtener los ranking de más vendidos y el revenue del mes anterior
22 ranked_sales AS (
23   SELECT
24     month,
25     product_name,
26     total_revenue,
27     total_quantity,
28     -- Ranking de cantidades por mes
29     RANK() OVER (PARTITION BY month ORDER BY total_quantity DESC) AS rank_in_month,
  
```

This query will process 343 B when run.

## ANEXOS

A) Poblado de tablas **raw** y variantes de models.



B) Historial de commits en repositorio de **GIT**.

